ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет Прикладной Информатики

Дисциплина: «Администрирование компьютерных сетей»

ОТЧЁТ

по лабораторной работе №3

Выполнили:

Демакова Ксения Александровна

Капленко Алина Денисовна

Саитов Камиль Булатович

Козлова Ульяна Сергеевна

Преподаватель:

Самохин Никита Юрьевич

Санкт-Петербург
2026

1 Цель работы и устройство стенда

Цель работы — развернуть кластер PostgreSQL с автоматическим выбором основного узла, проверить репликацию данных и доступ к базе через единый адрес HAProxy. Дополнительно проверяется возвращение остановленного узла и получение им изменений, сделанных за время отсутствия.

В ходе эксперимента после остановки основного узла реплика автоматически стала лидером. Чтение и запись через HAProxy возобновились на новом лидере. После запуска прежний лидер присоединился как синхронная реплика и получил все четыре тестовые записи.

1.1 Компоненты

Компонент	Назначение
PostgreSQL 15	Хранение данных и потоковая репликация WAL между двумя узлами.
Patroni	Управление PostgreSQL, ролями узлов и переключением лидера.
ZooKeeper 7.7.1	Хранение координационного состояния кластера и блокировки лидера.
HAProxy 3.0	Проксирование TCP-подключений к текущему основному узлу.

Patroni использует ZooKeeper как DCS. Данные PostgreSQL передаются непосредственно между узлами; ZooKeeper не переносит таблицы. HAProxy проверяет REST API Patroni на порту 8008 и направляет SQL-соединения на порт 5432 подходящего узла.

1.2 Среда выполнения и адреса

Работа выполнена локально в Windows с Docker Desktop и WSL 2. В выводе команд: WSL 3.0.3.0, Docker Compose 5.6.1; HAProxy 3.0.30.

Сервис	Адрес внутри Docker	Адрес с Windows
pg-master	pg-master:5432	127.0.0.1:5433
pg-slave	pg-slave:5432	127.0.0.1:5434
haproxy	haproxy:5432	127.0.0.1:5432
Статистика HAProxy	haproxy:7000	http://127.0.0.1:7000/
ZooKeeper	zoo:2181	Не опубликован

2 Развёртывание PostgreSQL и Patroni

Подготовлен образ lab03-postgres:patroni на базе postgres:15-bookworm. Patroni установлен в Python virtualenv вместе с поддержкой ZooKeeper. Конфигурации postgres0.yml и postgres1.yml копируются в образ. PostgreSQL запускается под управлением Patroni от пользователя postgres. Полные конфигурации приведены в приложении А.

В Docker Compose заданы два узла PostgreSQL с отдельными именованными томами и сервис zoo. Для обоих участников указан scope: my_cluster; включён synchronous_mode. REST API доступен внутри сети Docker, а внешние SQL-порты привязаны к 127.0.0.1.

```
docker compose up -d --build
docker compose ps
docker compose exec pg-master patronictl -c /postgres0.yml list
```

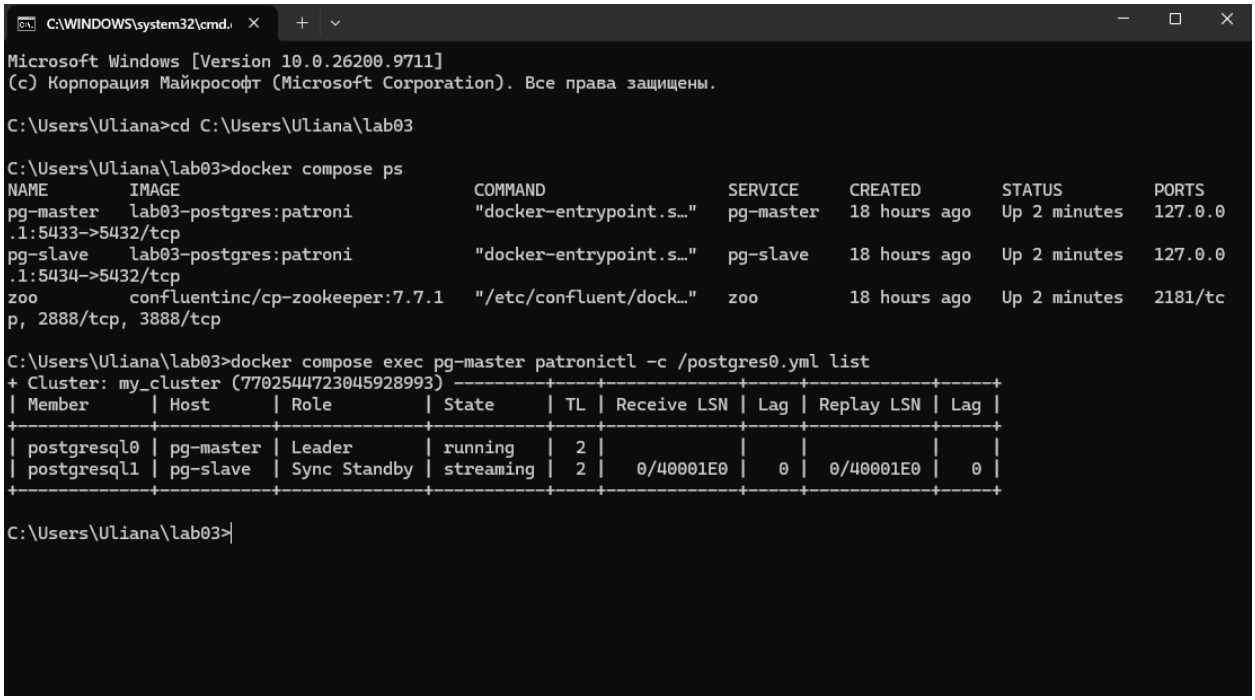


Рисунок 1 — Запущенные контейнеры и распределение ролей до эксперимента

Проверка показала: postgresql0 — Leader / running, postgresql1 — Sync Standby / streaming. Перед экспериментом оба узла находились на timeline 2, отставание реплики составляло 0.

2.1 Проверка ZooKeeper

```
docker compose logs --tail=50 zoo
```

В журнале ZooKeeper зафиксированы запуск сервера, привязка к порту 2181 и создание журнала транзакций.

```
INFO binding to port 0.0.0.0/0.0.0.0:2181
INFO PrepRequestProcessor (sid:0) started, reconfigEnabled=false
INFO Creating new log file: log.1
```

3 Проверка репликации и запрета записи

На основном узле создана таблица lab_students с первичным ключом id и текстовым полем name.

```
CREATE TABLE lab_students (id INTEGER PRIMARY KEY, name TEXT NOT NULL);
INSERT INTO lab_students VALUES (1, 'Uliana'), (2, 'Kamil');
```

SQL выполнялся через psql внутри pg-master. Параметр -h /tmp задаёт каталог Unix-сокета, указанный в конфигурации Patroni.

```
docker compose exec pg-slave psql -h /tmp -U postgres -d postgres -P pager=off ^
  -c "SELECT * FROM lab_students ORDER BY id;"
```

На реплике без повторного создания таблицы появились обе строки. Затем выполнена попытка изменения данных непосредственно на реплике:

```
docker compose exec pg-slave psql -h /tmp -U postgres -d postgres ^
  -c "INSERT INTO lab_students VALUES (3, 'Test');"
```

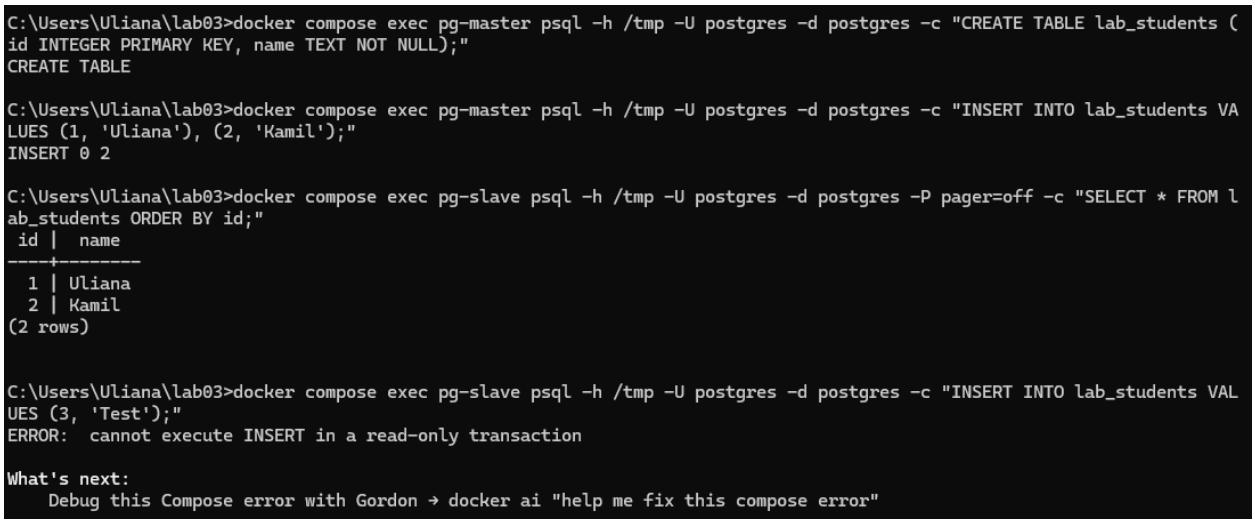


Рисунок 2 — Создание данных на лидере, чтение с реплики и отказ при записи на реплику

Ответ ERROR: cannot execute INSERT in a read-only transaction подтверждает режим только для чтения. Запись с именем Test не была добавлена. Таким образом, реплика получает изменения лидера, но не принимает обычные пользовательские операции записи до повышения до основного узла.

4 Настройка единой точки подключения

В Compose добавлен сервис haproxy с образом haproxy:3.0 и именем контейнера postgres_entrypoint. Порт 5432 используется для SQL-подключений, порт 7000 — для страницы статистики. Файл haproxy.cfg подключён как bind mount в режиме только для чтения.

```
docker compose config --quiet
docker compose up -d haproxy
docker compose ps
```

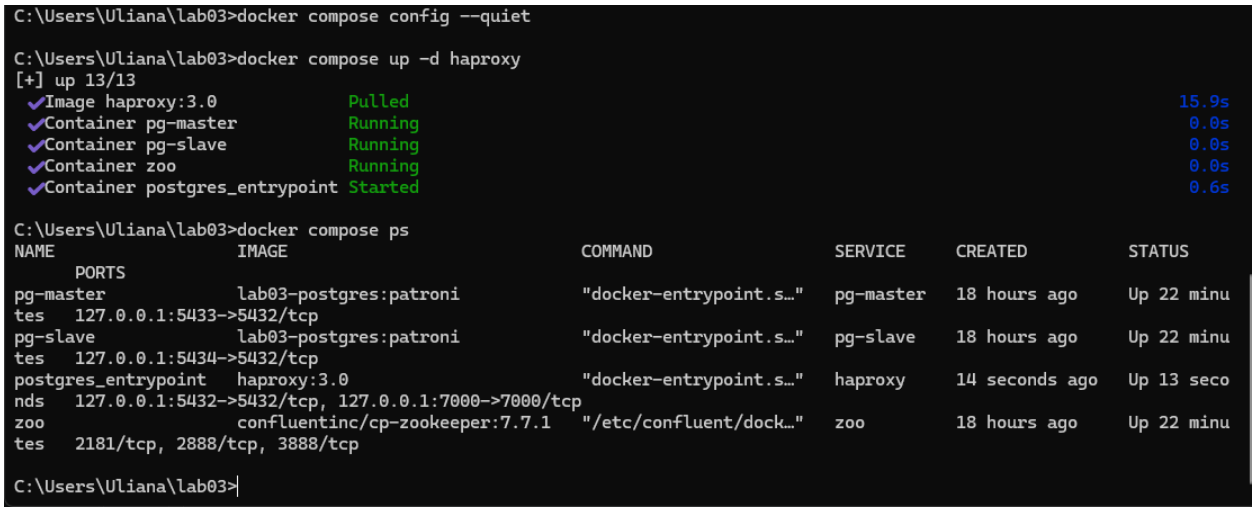


Рисунок 3 — Запуск HAProxy и публикация портов 5432 и 7000

4.1 Выбор основного узла

```
listen postgres
   bind *:5432
   option httpchk GET /primary
   http-check expect status 200
   default-server inter 3s fall 3 rise 2 on-marked-down shutdown-sessions
   server postgresql0 pg-master:5432 maxconn 100 check port 8008
   server postgresql1 pg-slave:5432 maxconn 100 check port 8008
```

Запрос GET /primary возвращает HTTP 200 только для работающего основного узла, владеющего блокировкой лидера. Поэтому в пул подключений включается один узел. Проверка обращается к REST API на порту 8008, а пользовательский трафик передаётся на порт PostgreSQL 5432.

Проверки выполняются с интервалом 3 секунды. Три неудачи подряд исключают узел; два успешных ответа возвращают его в пул. При исключении узла параметр shutdown-sessions завершает связанные с ним сеансы.

5 Чтение и запись через HAProxy

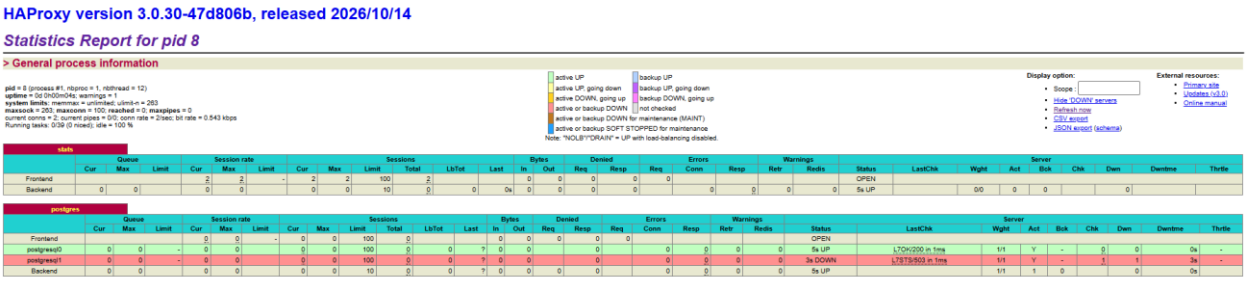


Рисунок 4 — Статистика HAProxy до остановки основного узла

Участник	Статус	Результат проверки
postgresql0	UP	L7OK / 200
postgresql1	DOWN	L7STS / 503

Для SQL-проверок использован psql из контейнера pg-slave с параметрами -h haproxy -p 5432. Контейнер запускает клиент; целевой сервер выбирается HAProxy. Пароль postgres передан через переменную PGPASSWORD.

```
C:\Users\Uliana\lab03>docker compose exec -e PGPASSWORD=postgres pg-slave psql -h haproxy -p 5432 -U postgres -d postgre
s -P pager=off -c "SELECT inet_server_addr(), pg_is_in_recovery();"
 inet_server_addr | pg_is_in_recovery
------------------+-------------------
 172.18.0.4       | f
(1 row)


C:\Users\Uliana\lab03>docker compose exec -e PGPASSWORD=postgres pg-slave psql -h haproxy -p 5432 -U postgres -d postgre
s -c "INSERT INTO lab_students VALUES (3, 'HAProxy test');"
INSERT 0 1

C:\Users\Uliana\lab03>docker compose exec -e PGPASSWORD=postgres pg-slave psql -h haproxy -p 5432 -U postgres -d postgre
s -P pager=off -c "SELECT * FROM lab_students ORDER BY id;"
 id |     name
----+--------------
  1 | Uliana
  2 | Kamil
  3 | HAProxy test
(3 rows)


C:\Users\Uliana\lab03>
```

Рисунок 5 — Чтение и запись через HAProxy до отказа

Запрос SELECT inet_server_addr(), pg_is_in_recovery() вернул адрес 172.18.0.4 и f: подключение попало на основной узел. Через тот же адрес haproxy успешно добавлена строка (3, HAProxy test), после чего SELECT показал три записи.

6 Остановка лидера и автоматическое переключение

Перед остановкой повторно проверены роли. Узел postgresql0 был лидером, postgresql1 — синхронной репликой, отставание составляло 0. Затем выполнена остановка контейнера с текущим лидером:

```
docker compose exec pg-slave patronictl -c /postgres1.yml list
docker compose stop pg-master
docker compose exec pg-slave patronictl -c /postgres1.yml list
docker compose logs --tail=60 pg-slave
```

После остановки patronictl показал единственного активного участника postgresql1 в роли Leader / running с timeline 3. На странице HAProxy активный узел также изменился.

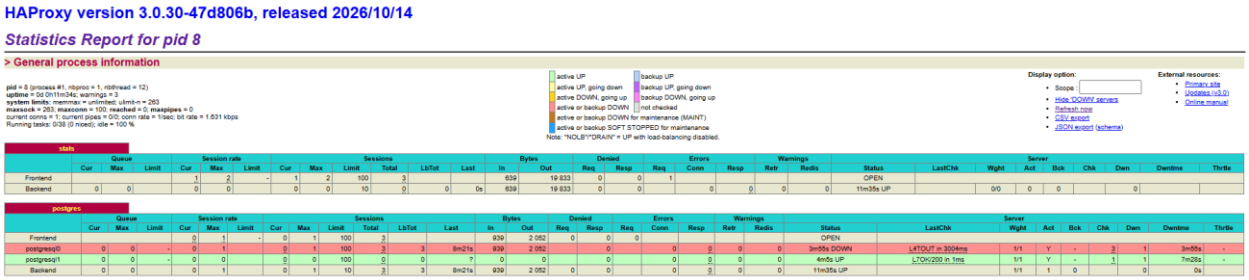


Рисунок 6 — HAProxy после остановки pg-master: postgresql1 стал активным узлом

Участник	Статус в HAProxy	Проверка
postgresql0	DOWN	L4TOUT
postgresql1	UP	L7OK / 200

6.1 Ключевые события журнала

Фрагменты журнала pg-slave:

```
14:32:20.080  replication terminated by primary server
14:32:20.085  could not connect to the primary server
14:32:21,060  promoted self to leader by acquiring session lock
14:32:21.062  received promote request
14:32:21.068  selected new timeline ID: 3
14:32:21.175  database system is ready to accept connections
14:32:22,097  I am (postgresql1), the leader with the lock
```

Сначала прекратилась репликация со старого лидера. Затем Patroni на втором узле получил блокировку лидера, инициировал promotion и завершил восстановление PostgreSQL. HAProxy обнаружил HTTP 200 на /primary нового лидера и включил его в пул.

7 Проверка работы после переключения

При выключенном pg-master повторно выполнены запросы через тот же адрес haproxy:5432.

```
docker compose exec -e PGPASSWORD=postgres pg-slave psql -h haproxy -p 5432 -U ^
  postgres -d postgres -P pager=off ^
  -c "SELECT inet_server_addr(), pg_is_in_recovery(); SELECT * FROM lab_students ORDER BY id;"
```

Вывод терминала:

```
 inet_server_addr | pg_is_in_recovery
------------------+-------------------
 172.18.0.2       | f
(1 row)

 id |     name
----+--------------
  1 | Uliana
  2 | Kamil
  3 | HAProxy test
(3 rows)
```

Адрес целевого сервера изменился с 172.18.0.4 на 172.18.0.2. Значение f подтверждает, что соединение направлено на основной узел.

```
docker compose exec -e PGPASSWORD=postgres pg-slave psql -h haproxy -p 5432 -U ^
  postgres -d postgres ^
  -c "INSERT INTO lab_students VALUES (4, 'After failover');"

INSERT 0 1

docker compose exec -e PGPASSWORD=postgres pg-slave psql -h haproxy -p 5432 -U ^
  postgres -d postgres -P pager=off ^
  -c "SELECT * FROM lab_students ORDER BY id;"

 id |      name
----+----------------
  1 | Uliana
  2 | Kamil
  3 | HAProxy test
  4 | After failover
(4 rows)
```

Успешная вставка и последующее чтение подтверждают доступность операций записи на новом лидере при остановленном прежнем лидере.

8 Возвращение прежнего лидера

После проверки записи остановленный контейнер запущен без удаления томов и без ручной смены ролей:

```
docker compose start pg-master
docker compose exec pg-slave patronictl -c /postgres1.yml list
```

В итоговом выводе patronictl зафиксировано следующее состояние:

Участник	Роль	Состояние	TL	Отставание
postgresql0 pg-master	Sync Standby	streaming	3	0
postgresql1 pg-slave	Leader	running	3	-

У реплики Receive LSN и Replay LSN совпали: 0/7000C48. Прежний лидер присоединился к работающему основному узлу.

```
docker compose exec pg-master psql -h /tmp -U postgres -d postgres -P pager=off ^
  -c "SELECT pg_is_in_recovery(); SELECT * FROM lab_students ORDER BY id;"
```

Вывод на вернувшемся узле:

```
 pg_is_in_recovery
-------------------
 t
(1 row)

 id |      name
----+----------------
  1 | Uliana
  2 | Kamil
  3 | HAProxy test
  4 | After failover
(4 rows)
```

Значение t подтверждает работу pg-master в режиме реплики. Строка After failover была создана во время его остановки и появилась после возвращения. Это подтверждает автоматическое получение пропущенных изменений.

9 Результаты и вывод

Проверка	Наблюдаемый результат
Развёртывание кластера	Два узла PostgreSQL и ZooKeeper запущены; определены лидер и синхронная реплика.
Репликация	Таблица и две исходные строки появились на реплике автоматически.
Запись на реплику	INSERT отклонён с ошибкой read-only transaction.
Единый адрес	Чтение и запись через haproxy:5432 успешны.
Остановка лидера	postgresql1 автоматически стал лидером; timeline изменился с 2 на 3.
Работа после переключения	Через прежний адрес прочитаны старые строки и добавлена After failover.
Возвращение узла	postgresql0 стал Sync Standby; получил четыре строки; отставание 0.

В работе развёрнут кластер PostgreSQL с Patroni, ZooKeeper и HAProxy. Подтверждены потоковая репликация, ограничение записи на реплике, автоматическое повышение реплики после остановки лидера и маршрутизация новых подключений к актуальному основному узлу. После возвращения прежний лидер автоматически синхронизировался с новым.

Приложение А Конфигурации стенда

A1 Dockerfile

```
FROM postgres:15-bookworm
RUN apt-get update && \
    apt-get install -y python3-venv python3-pip curl \
    netcat-openbsd iputils-ping && \
    rm -rf /var/lib/apt/lists/*
RUN python3 -m venv /opt/patroni-venv && \
    /opt/patroni-venv/bin/pip install --no-cache-dir \
    "patroni[zookeeper]" psycopg2-binary
ENV PATH="/opt/patroni-venv/bin:$PATH"
COPY postgres0.yml /postgres0.yml
COPY postgres1.yml /postgres1.yml
WORKDIR /var/lib/postgresql
USER postgres
```

A2 haproxy.cfg

```
global
    maxconn 100

defaults
    log global
    mode tcp
    retries 3
    timeout client 30m
    timeout connect 4s
    timeout server 30m
    timeout check 5s

listen stats
    mode http
    bind *:7000
    stats enable
    stats uri /

listen postgres
    bind *:5432
    option httpchk GET /primary
    http-check expect status 200
    default-server inter 3s fall 3 rise 2 on-marked-down shutdown-sessions
    server postgresql0 pg-master:5432 maxconn 100 check port 8008
    server postgresql1 pg-slave:5432 maxconn 100 check port 8008
```

A3 docker-compose.yml

```
services:
  pg-master:
    build: .
    image: lab03-postgres:patroni
    container_name: pg-master
    hostname: pg-master
    restart: unless-stopped
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      PGDATA: /var/lib/postgresql/data/pgdata
    expose:
      - "8008"
    ports:
      - "127.0.0.1:5433:5432"
    volumes:
      - pg-master:/var/lib/postgresql/data
    command: patroni /postgres0.yml
    depends_on:
      - zoo

  pg-slave:
    build: .
    image: lab03-postgres:patroni
    container_name: pg-slave
    hostname: pg-slave
    restart: unless-stopped
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      PGDATA: /var/lib/postgresql/data/pgdata
    expose:
      - "8008"
    ports:
      - "127.0.0.1:5434:5432"
    volumes:
      - pg-slave:/var/lib/postgresql/data
    command: patroni /postgres1.yml
    depends_on:
      - zoo
```

Продолжение этого же файла — на следующей странице.

A3 docker-compose.yml продолжение

```
  zoo:
    image: confluentinc/cp-zookeeper:7.7.1
    container_name: zoo
    hostname: zoo
    restart: unless-stopped
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  haproxy:
    image: haproxy:3.0
    container_name: postgres_entrypoint
    restart: unless-stopped
    ports:
      - "127.0.0.1:5432:5432"
      - "127.0.0.1:7000:7000"
    depends_on:
      - pg-master
      - pg-slave
      - zoo
    volumes:
      - ./haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro

volumes:
  pg-master:
  pg-slave:
```

Сервисы подключаются к общей сети, создаваемой Compose. Именованные тома pg-master и pg-slave сохраняют данные при остановке и пересоздании контейнеров. Директива depends_on задаёт порядок запуска, но в этой короткой форме не проверяет готовность PostgreSQL или ZooKeeper.

Пароли для PostgreSQL, которым управляет Patroni, явно заданы в разделе postgresql.authentication файлов ниже. Параметр data_dir Patroni указывает фактический каталог данных каждого участника внутри его тома.

A4 postgres0.yml

```
scope: my_cluster
name: postgresql0
restapi:
  listen: 0.0.0.0:8008
  connect_address: pg-master:8008
zookeeper:
  hosts:
    - zoo:2181
bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 10485760
    synchronous_mode: true
    postgresql:
      use_pg_rewind: true
      use_slots: true
      parameters:
        wal_level: replica
        hot_standby: "on"
        wal_keep_size: 128MB
        max_wal_senders: 10
        max_replication_slots: 10
        wal_log_hints: "on"
        archive_mode: "always"
        archive_timeout: 1800s
        archive_command: >-
          mkdir -p /tmp/wal_archive &&
          test ! -f /tmp/wal_archive/%f &&
          cp %p /tmp/wal_archive/%f
  initdb:
    - encoding: UTF8
    - data-checksums
  pg_hba:
    - local all all trust
    - host replication replicator 0.0.0.0/0 md5
    - host all all 0.0.0.0/0 md5
postgresql:
  listen: 0.0.0.0:5432
  connect_address: pg-master:5432
  data_dir: /var/lib/postgresql/data/postgresql0
  bin_dir: /usr/lib/postgresql/15/bin
  pgpass: /tmp/pgpass0
  authentication:
    replication:
      username: replicator
      password: rep-pass
    superuser:
      username: postgres
      password: postgres
  parameters:
    unix_socket_directories: /tmp
watchdog:
  mode: "off"
tags:
  nofailover: false
  noloadbalance: false
  clonefrom: false
  nosync: false
```

A5 postgres1.yml

```
scope: my_cluster
name: postgresql1
restapi:
  listen: 0.0.0.0:8008
  connect_address: pg-slave:8008
zookeeper:
  hosts:
    - zoo:2181
bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 10485760
    synchronous_mode: true
    postgresql:
      use_pg_rewind: true
      use_slots: true
      parameters:
        wal_level: replica
        hot_standby: "on"
        wal_keep_size: 128MB
        max_wal_senders: 10
        max_replication_slots: 10
        wal_log_hints: "on"
        archive_mode: "always"
        archive_timeout: 1800s
        archive_command: >-
          mkdir -p /tmp/wal_archive &&
          test ! -f /tmp/wal_archive/%f &&
          cp %p /tmp/wal_archive/%f
  initdb:
    - encoding: UTF8
    - data-checksums
  pg_hba:
    - local all all trust
    - host replication replicator 0.0.0.0/0 md5
    - host all all 0.0.0.0/0 md5
postgresql:
  listen: 0.0.0.0:5432
  connect_address: pg-slave:5432
  data_dir: /var/lib/postgresql/data/postgresql1
  bin_dir: /usr/lib/postgresql/15/bin
  pgpass: /tmp/pgpass1
  authentication:
    replication:
      username: replicator
      password: rep-pass
    superuser:
      username: postgres
      password: postgres
  parameters:
    unix_socket_directories: /tmp
watchdog:
  mode: "off"
tags:
  nofailover: false
  noloadbalance: false
  clonefrom: false
  nosync: false
```

10 Ответы на вопросы задания

10.1 Чем отличаются expose и ports

Директива expose указывает внутренние порты сервиса и не публикует их на Windows-хосте. Контейнеры одной сети Docker могут обращаться к слушающим портам друг друга и без expose: эта директива не служит межконтейнерным firewall. Директива ports создаёт публикацию порта контейнера на хосте

В данной работе expose: 8008 обозначает REST API Patroni для HAProxy. Запись ports: "127.0.0.1:5433:5432" означает, что подключение с Windows к 127.0.0.1:5433 направляется на порт 5432 контейнера pg-master. Внутри Docker используется pg-master:5432. Публикация с привязкой к 127.0.0.1 рассчитана на локальные подключения с хоста.

Если поместить 8008 в ports и задать публикацию, к API можно будет обращаться и через соответствующий порт хоста. Для работы HAProxy внутри сети это не требуется. Поэтому 8008 не опубликован; ZooKeeper также доступен только внутри сети проекта.

10.2 Пересобирается ли образ при перезапуске

Команды docker compose restart и docker compose stop/start перезапускают существующие контейнеры и не выполняют сборку образа. Изменение файлов на диске само по себе не изменяет уже собранный образ.

Изменение	Что требуется
Файлы не менялись	Обычный перезапуск использует существующие контейнеры и образ.
Изменён postgres0.yml или postgres1.yml	Они добавлены через COPY. Нужны пересборка образа и пересоздание контейнеров.
Изменён Dockerfile	Нужна пересборка и применение нового образа; неизменившиеся слои могут быть взяты из кэша.
Изменён haproxy.cfg	Это bind mount. Пересборка образа не нужна; после проверки конфигурации нужен reload или restart HAProxy.

Для явного применения изменений Dockerfile и файлов COPY используется команда

```
docker compose up -d --build pg-master pg-slave
```

Отдельно учитывается конфигурация Patroni: bootstrap.dcs задаёт начальные параметры кластера. Для уже созданного кластера динамические настройки меняются через patronictl edit-config или REST API